



INFORMATION
TECHNOLOGY
UNIVERSITY

**Computer Programming Fundamentals
Lab 08**

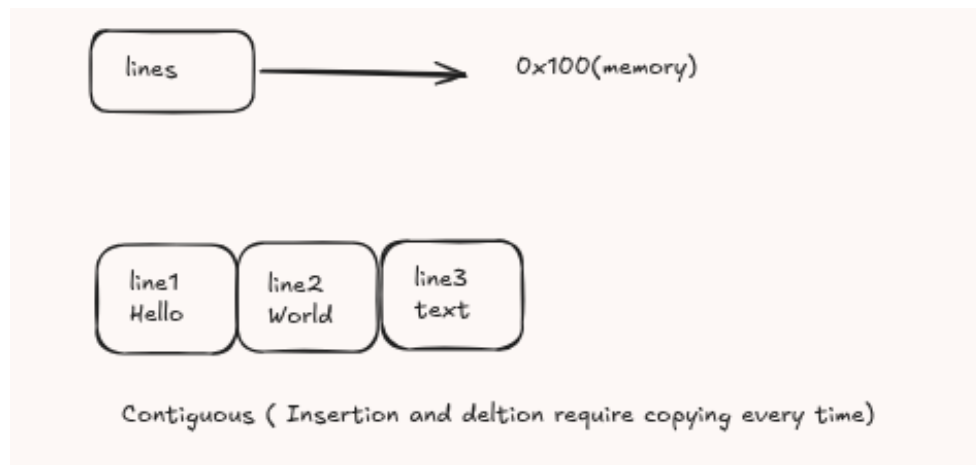
**Instructor: Ahmad Abd eMuneeb Niazi
Session: Fall 2025**

Question 01:

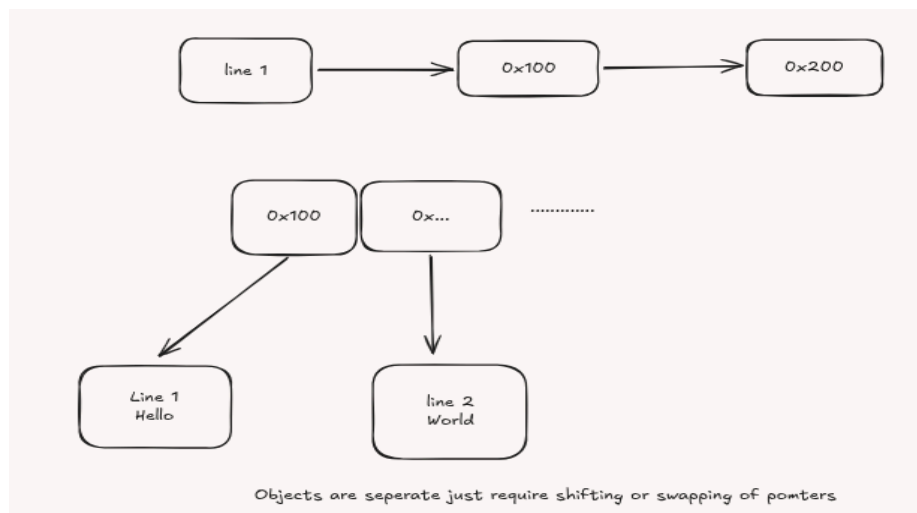
Objective

The goal of this assignment is to implement a **console-based text editor** in C++ using **Object-Oriented Programming** concepts and **dynamic memory management**. Instead of storing lines of text as objects in a contiguous array (`MyString*`), you will use a **double pointer (`MyString**`)**. This approach stores pointers to individual `MyString` objects, allowing for efficient insertion and deletion without copying entire objects. The assignment is designed to teach efficient memory management, pointer manipulation, and basic text editor operations such as insert, delete, and display.

Previous Implementation:



Implementation Required:



Concept

In the old approach, a document stored lines as a single array of MyString objects. This meant that inserting or deleting a line required **copying the entire object**, which can be expensive for large text. With MyString**, the document stores **pointers to each line**, and each MyString object resides separately in heap memory. When inserting or deleting, only the pointers are shifted, not the actual objects, which reduces memory operations and increases efficiency.

Memory layout for MyString** looks like this: the lines array holds pointers, and each pointer points to a MyString object stored elsewhere in the heap. This structure allows dynamic growth and flexible management of lines.

Class Design

The Document class should have the following members:

- MyString** lines — pointer to an array of pointers to MyString objects.
- int size — number of lines currently in the document.

The Document class should implement:

1. **Constructor** — initialize the lines array to hold the first line.
2. **Destructor** — delete all MyString objects and then the pointer array.
3. **Insert Line** — insert a new line at a given index, shifting only pointers.
4. **Delete Line** — remove a line and shift remaining pointers.
5. **Display** — print the full document.
6. **Get Line** — access a specific line by returning its pointer.

Question 02:

Task 2: names.txt → sorted_names.txt

Objective Read names from a file (names.txt) using Mystr* or Mystr** and generate a new file (sorted_names.txt) that contains all names sorted in alphabetical order.

Requirements

- Read all names dynamically from names.txt.
- Store them using dynamic memory (Mystr* or Mystr**).
- Sort the names using any sorting algorithm (e.g., bubble sort, selection sort etc).

- Write the sorted names into a new file named sorted_names.txt.

Before sorting

Memory structure using MyStr**

After reading file:

names (MyStr**)

```

    |
    v
+-----+-----+-----+-----+
| ptr   | ptr   | ptr   | ptr   |
+-----+-----+-----+-----+
|       |       |       |       |
v       v       v       v
"Zara"  "Ali"  "Ahmed" "Usman"

```

Then sort it and stores in the sorted.txt file

Question 03:

Task 3: Read student.txt and generate sorted_student.txt

You are required to read student data from a file and generate a sorted output file. Each student record contains a name, roll number, CGPA, and a list of course marks.

class Student {

Mystr name;

int roll_num;

float cgpa;

MyVector* courses;

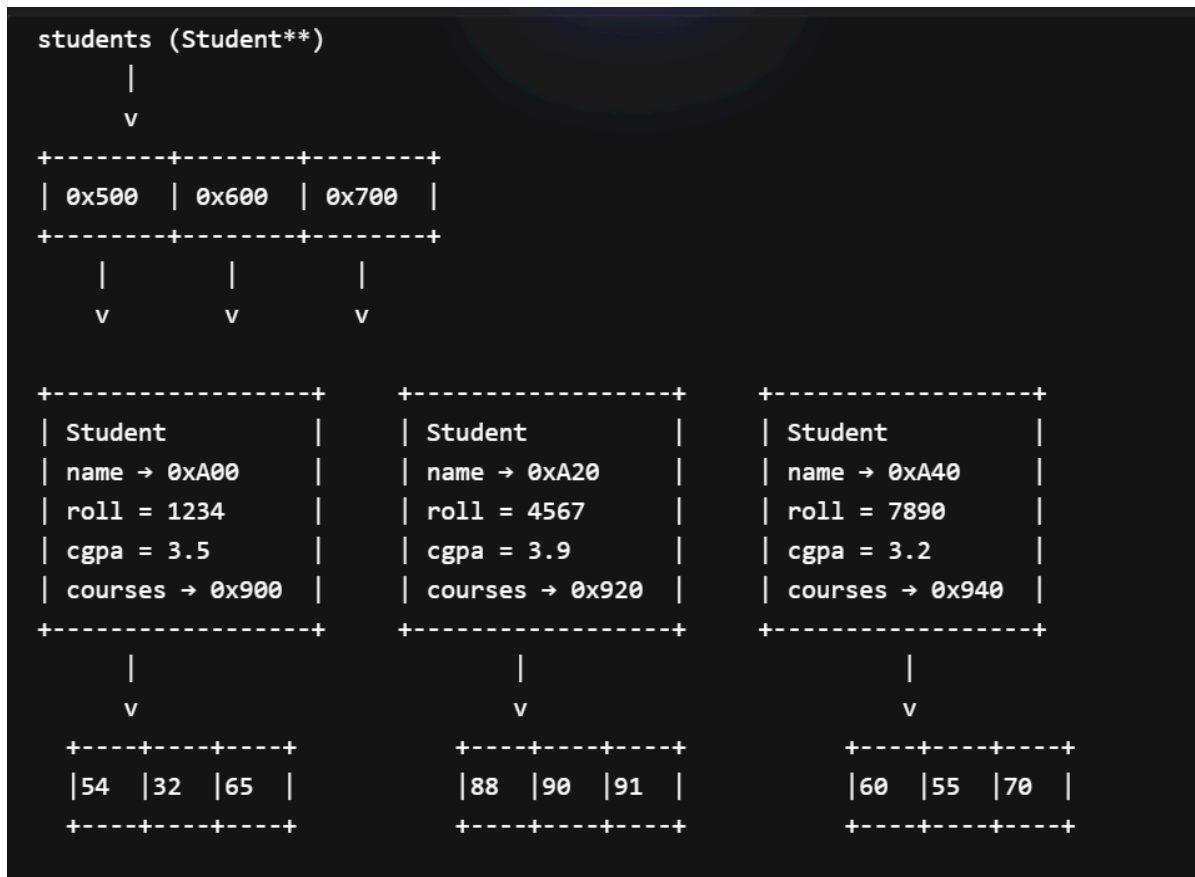
}

Input File Format (student.txt) Each line of the file will contain data in the following format:

ali ahmed, 1234, 3.5, **54,32,65,76,80**

Requirements

- Read all student records dynamically from student.txt.
- Store them in a dynamic structure (e.g., array of Student objects using pointers).
- Sort students based on name or roll number (as instructed by the teacher).
- Write the sorted records into a new file named sorted_student.txt in the same format as the input file.
- Properly manage dynamic memory for Mystr and MyVector objects to avoid memory leaks.



GOOD LUCK !